



Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Архітектура Комп'ютерів. Частина 1

Лабораторна робота №3

«Перетворення даних в ЕОМ на мікропрограмному рівні»

Виконав: студент 2 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Верба О. А.*

Тема: «Перетворення даних в ЕОМ на мікропрограмному рівні» .

Мета: вивчити архітектуру ЕОМ, що містить блок мікропрограмного керування і арифметико-логічний пристрій з зосередженою логікою та двухадресним надоперативним запам'ятовуючим пристроєм (НОЗП), одержати навички розробки мікропрограм.

Порядок виконання роботи

1. Визначити індивідуальний варіант за номером залікової книжки: встановити значення розрядів $a_7, a_6, \dots, a_1$, вибрати спосіб множення, арифметичну операцію, форму подання чисел та значення операндів X і Y .
2. Проаналізувати задану структуру ЕОМ, що містить блок мікропрограмного керування, арифметико-логічний пристрій із зосередженою логікою та двоадресний НОЗП.
3. Визначити ознаку g за відповідним розрядом регістра стану процесора та сформулювати функцію, яку необхідно реалізувати для випадків $g = 1$ і $g = 0$.
4. Побудувати операційні схеми для множення чисел першим способом та для обчислення арифметичної функції $2^2X - 2^{-1}Y$.
5. Розробити функціональні мікроалгоритми виконання заданих перетворень даних на мікропрограмному рівні.
6. Скласти таблиці станів регістрів для режиму множення та для режиму обчислення арифметичної функції, враховуючи форми подання операндів і результату.
7. Записати мікропрограму мовою мікроасемблера відповідно до побудованого мікроалгоритму.
8. Виконати моделювання роботи мікропрограми для різних значень ознаки g та порівняти отримані результати з ручними обчисленнями.
9. Сформулювати висновки щодо правильності розробленого мікроалгоритму та досягнення мети лабораторної роботи.

Хід роботи

Мій номер залікової книжки: 4106, що у двіковому вигляді є 0001 0000 0000 1010. Звідси $a_7 = 0, a_6 = 0, a_5 = 0, a_4 = 1, a_3 = 0, a_2 = 1, a_1 = 0$. Тобто я маю такий варіант:

a_5	a_4	Спосіб множення (при $g = 1$)
0	0	4
0	1	1
1	0	2
1	1	3

(а) Спосіб множення

a_6	a_1	Арифметична операція (при $g = 0$)
0	0	$2^2X - 2^{-1}Y$
0	1	$2^{-1}X + 2^2Y$
1	0	$2^3X - 2Y$
1	1	$2^2X + 2^{-1}Y$

(б) Арифметична операція

a_3	a_2	X	Y	Z
0	0	ДК	ДК	ПК
0	1	ПК	ПК	ДК
1	0	ПК	ДК	ПК
1	1	ДК	ПК	ДК

(в) Форма подання чисел

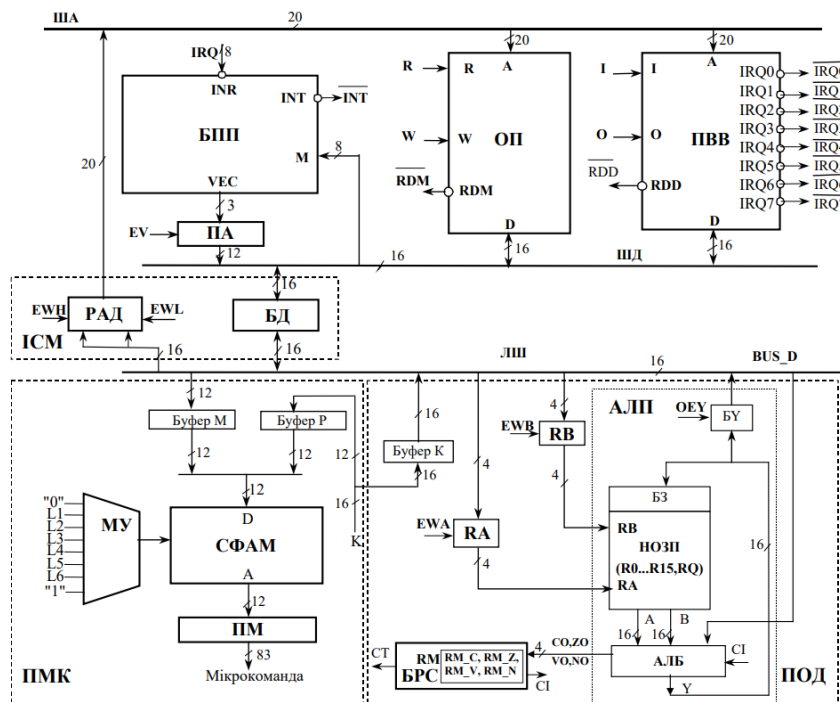
a_7	a_1	X	Y
0	0	7	-5
0	1	-7	9
1	0	-9	-5
1	1	7	-6

(г) Значення операндів

Табл. 1: Таблиці вибору функцій і тестових даних

А ознака g визначається у вихідному стані значенням розряду з двійковим номером $a_3a_2a_1 = 010 = 2$ регістра <RN> стану процесора.

Загальна структура EOM:



Надам операційні схеми для функції

$$Z = \begin{cases} X \cdot Y, & \text{якщо } g = 1, \\ 2^2 X - 2^{-1} Y, & \text{якщо } g = 0. \end{cases}$$

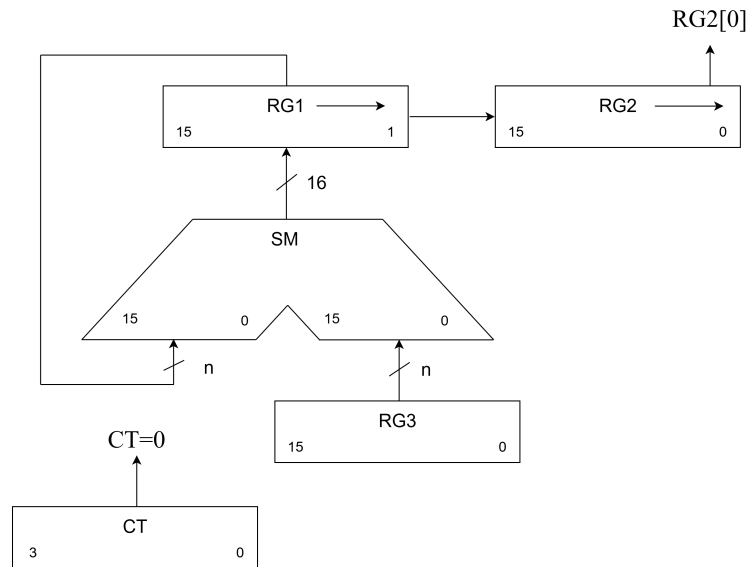


Рис. 1: Операційна схема множення 1 способом

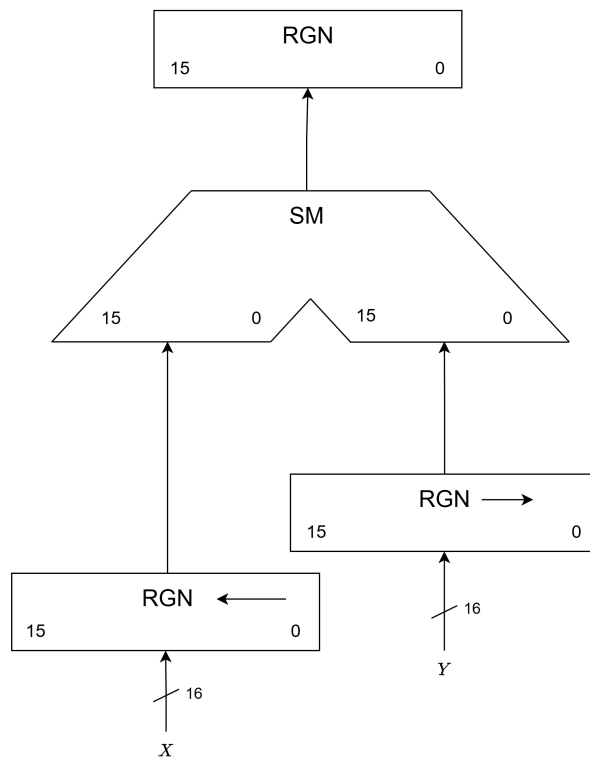
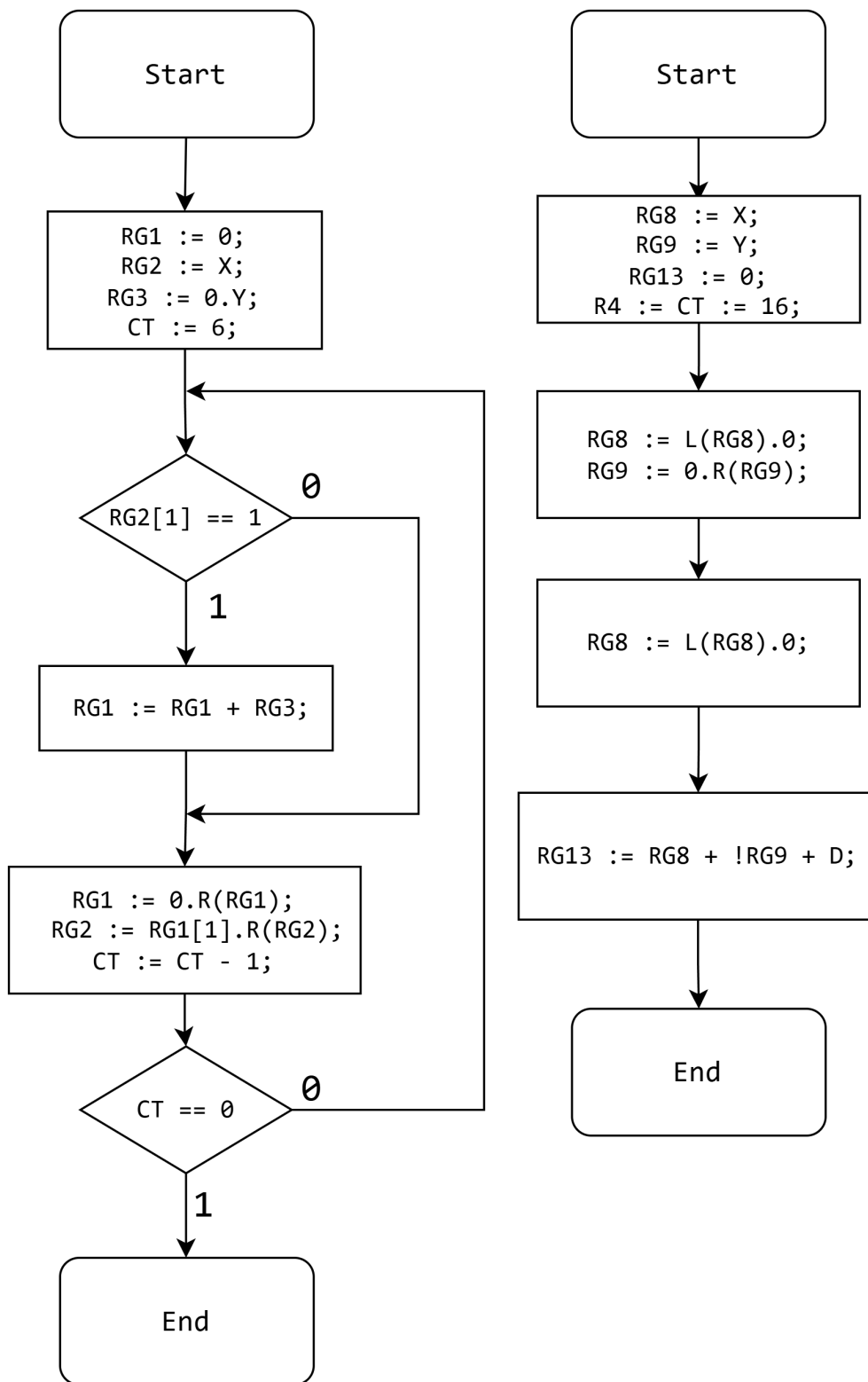


Рис. 2: Операційна схема функції $2^2 X - 2^{-1} Y$

Наведемо функціональні мікроалгоритми для обчислення функцій:



Таблиця станів реєстрів при $g = 1$ (множення 1 способом):

Таблиця станів регістрів при $g = 0$ (обчислення функції $2^2X - 2^{-1}Y$):

Число $X = 7$ в ДК: 00.00000000000111, число $Y = -5$ в ДК: 11.1111111111011

№	RG8 15 - 0	RG9 15 - 0	RG13 15 - 0	Мікрооперації
0	0000000000000111	111111111111011	0000000000000000	RG13 := 0..00; RG8 := X; RG9 := Y;
1	0000000000000111 <i>Після зсуву:</i> 0000000000001110	111111111111011 <i>Після зсуву:</i> 11111111111101		RG8 := L(RG8).0 RG9 := 1.R(RG9)
2	0000000000001110 <i>Після зсуву:</i> 0000000000011100			RG8 := L(RG8).0
3			+0000000000011100 0000000000000010 0000000000000001 ----- 0000000000011111	RG13 := RG8 + !RG9 + + D;

Отримали число $Z = 00.00000000011111$ в ДК, що дорівнює 31 в десятковій, та 1F в шістнадцятковій системі числення.

Запишемо код мікроасемблера:

```

1  \z = x * y, g = 1
2  \z = 4x - 0.5y, g = 0
3  \x, y zadani v PK
4  \z u DK
5
6  \_____nalashtuvannya shemy_____
7  link L1:ct
8  link L2:rdm
9  link ewh:16
10
11 accept R14:0004h \g v 3-mu biti, g=1 dlya testu
12 accept R0:0005h  \adres x
13 accept R1:0008h  \adres z
14 accept R4:0010h  \CT := 16
15 accept R5:0000h  \znak x
16 accept R6:0000h  \znak y
17 accept R7:0000h  \znak z
18 accept R8:0000h  \temp
19 accept R9:0000h  \temp
20 accept R10:0000h \temp
21
22 accept R2:0000h  \x
23 accept R3:0000h  \y
24
25 accept R11:0000h \RES starshi rozryadi

```

```

26 accept R12:0000h \RES molodshi rozryadi
27
28 accept rdm_delay:3
29
30 dw 5h:00009h \x = 9 (PK)
31 dw 6h:0C005h \y = -5 (PK)
32 dw 8h:00000h \z
33
34 \_____zavantazheniya danyh z OP_____
35 {ewh; oey; xor nil, R0, R0;} \adres (19-16)
36 {ewl; oey; or nil, R0, z;} \adres x
37 {cjp rdm, cp; R; or R2, bus_d, z;} \R2 := x (PK)
38
39 {add R0, R0, 1, z;} \adres y = adres x + 1
40 {ewl; oey; or nil, R0, z;} \adres y
41 {cjp rdm, cp; R; or R3, bus_d, z;} \R3 := y (PK)
42
43 \_____vydilenniya znakov_____
44 sign_x
45 {and R5, R2, 0C000h; load RM, flags;} \znak x
46
47 sign_y
48 {and R6, R3, 0C000h; load RM, flags;} \znak y
49
50 \_____analiz g_____
51 analyze_g
52 {and nil, R14, 0004h; load RM, flags;} \analiz 3-go bita
53 {cjp RM_Z, FORM_PREP;} \yakshcho g = 0 -> formula
54
55 \=====
56 \ MNOZHENNYA: beremo moduli z PK
57 \=====
58 MULT_PREP
59 {and nil, R5, 0C000h; load RM, flags;} \x vidyemne?
60 {cjp RM_Z, mult_y_prep;}
61 {xor R2, R2, 0C000h;} \R2 := |x| z PK
62
63 mult_y_prep
64 {and nil, R6, 0C000h; load RM, flags;} \y vidyemne?
65 {cjp RM_Z, mult_sign;}
66 {xor R3, R3, 0C000h;} \R3 := |y| z PK
67
68 mult_sign
69 {or R7, R5;} \R7 := znak x
70 {xor R7, R6;} \R7 := znak rezultatu
71 {xor R11, R11, z;} \starshi rozryady = 0
72 {xor R12, R12, z;} \molodshi rozryady = 0
73 {load RM, z;} \obnulenniya oznak
74
75 \operatsiya mnozheniya moduliv
76 {or srl, nil, R2, z;} \otrymaly molodshyy bit mnozhnyka
77
78 M1
79 {cjp not RM_C, M2;} \R2[1] = 1?

```



```

80 {add R11, R11, R3, z;} \R11 := R11 + R3
81
82 M2
83 {or srl, R11, z;} \R11 := 0.R(R11)
84 {or sr.9, R2, z;} \R2 := R11[1].R(R2)
85 {or srl, nil, R2, z;} \znovu otrymaly R2[1]
86 {sub R4, R4, z, z; load RM, flags; cem_c;} \CT := CT - 1
87 {cjp not RM_Z, M1;} \CT = 0?
88
89 {or R12, R2, z;} \molodshi rozryady rezultatu
90
91 \_____nakladannya znaku na rezultat mnozhennya_____
92 {and nil, R7, 0C000h; load RM, flags;}
93 {cjp RM_Z, RES1;} \yakshcho rezultat dodatniy
94
95 \vidyemnyy rezultat -> perehid do DK
96 {xor R12, R12, 3FFFh;}
97 {add R12, R12, z, nz;} \+1
98 {or R11, 0FFFFh, z;} \sign extension dlya starshoho slova
99
100 \_____zapys rezultatu mnozhennya_____
101 RES1
102 {ewh; oey; xor nil, R1, R1;} \adres (19-16)
103 {ewl; oey; or nil, R1, z;} \adres z
104 {cjp rdm, cp; W; or nil, R12, z; oey;} \z -> OP
105 {cjp nz, END;}
106
107 \=====
108 \ FORMULA: peretvoryuyemo X,Y z PK u DK
109 \=====
110 FORM_PREP
111 {and nil, R5, 0C000h; load RM, flags;} \x vidyemne?
112 {cjp RM_Z, form_y_prep;}
113 {xor R2, R2, 0C000h;} \znyaly znakovI bity
114 {sub R2, z, R2, nz;} \R2 := x v DK
115
116 form_y_prep
117 {and nil, R6, 0C000h; load RM, flags;} \y vidyemne?
118 {cjp RM_Z, FORM;}
119 {xor R3, R3, 0C000h;} \znyaly znakovI bity
120 {sub R3, z, R3, nz;} \R3 := y v DK
121
122 \_____obchyslennya formuly_____
123 FORM
124 {or sll, R2, R2, z;} \2x
125 {or sll, R2, R2, z;} \4x
126 {or sra, R3, R3, z;} \0.5y
127
128 {or R12, R2, z;} \R12 := 4x
129 {sub R12, R12, R3, nz;} \R12 := 4x - 0.5y
130
131 \_____zapys rezultatu formuly_____
132 RES2
133 {ewh; oey; xor nil, R1, R1;} \adres (19-16)

```

```

134 {ewl; oey; or nil, R1, z;} \adres z
135 {cjp rdm, cp; W; or nil, R12, z; oey;} \z -> OP
136
137 END
138 {} \end

```

Та продемонструємо вивід програми (при різних режимах g):

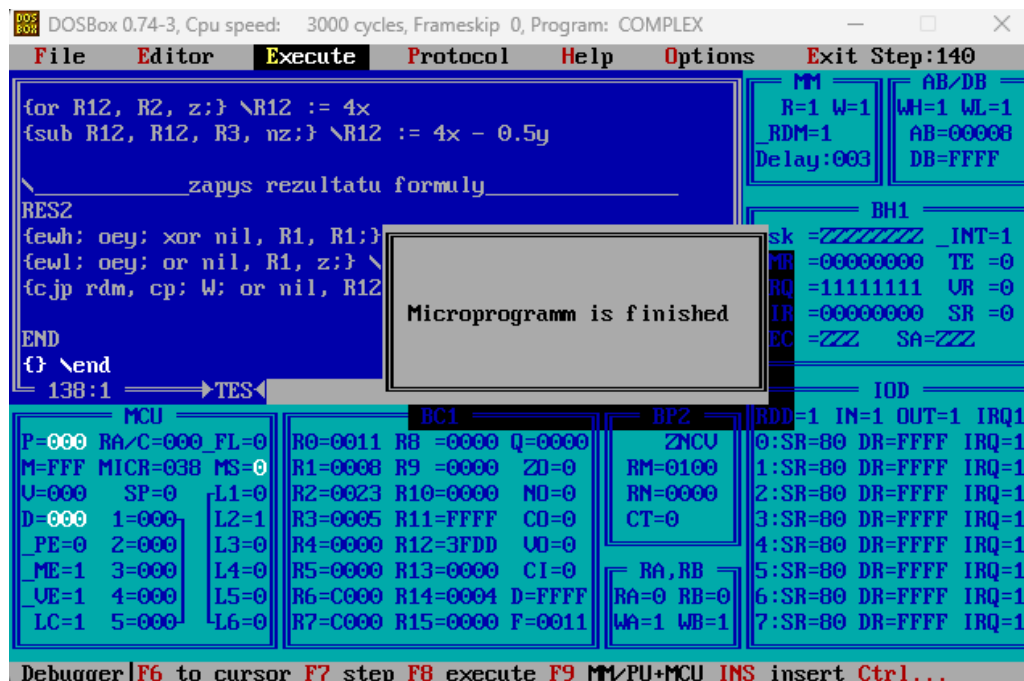


Рис. 3: Результат множення

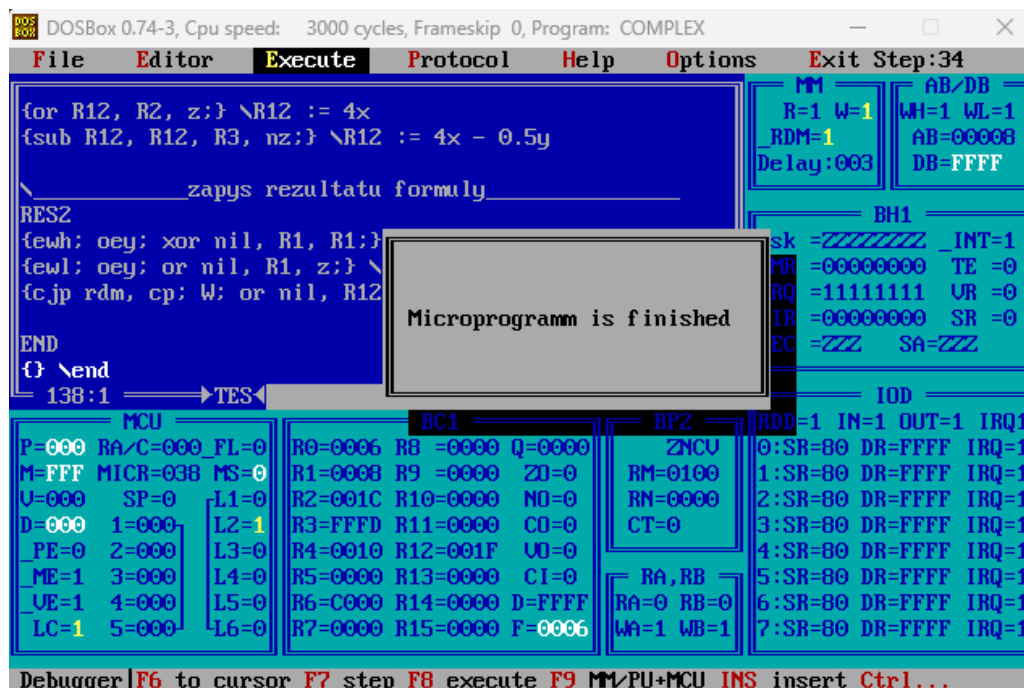


Рис. 4: Результат обчислення функції

Для множення, відповідь вийшла FFFF 3FDD, що дорівнює значенню FFFF FFDD, раніше обчисленим (тут просто особливість маски на другому регістрі), а під час обчи-

слення функції відповідь дорівнює 001F, що також співпадає із раніше обчисленим. Всі ці числа вже подані в ДК. Як бачимо, результати співпадають.

Висновок

У ході виконання лабораторної роботи було досліджено перетворення даних в ЕОМ на мікропрограмному рівні та закріплено принципи роботи архітектури, що містить блок мікропрограмного керування, арифметико-логічний пристрій із зосередженою логікою та двоадресний НОЗП. Для заданого варіанта було визначено функцію обчислення, форму подання операндів і результату, а також спосіб множення: при $g = 1$ реалізується множення $X \cdot Y$ першим способом, а при $g = 0$ обчислюється вираз $2^2X - 2^{-1}Y$.

У процесі роботи було побудовано операційні схеми та функціональний мікроалгоритм для обох режимів роботи, складено таблиці станів регістрів і розроблено мікропрограму для виконання заданих перетворень. Окрему увагу було приділено аналізу знаків операндів, переходу між прямим і додатковим кодами, виконанню зсувів, додавання, віднімання та запису результату до оперативної пам'яті у додатковому коді.

Результати моделювання підтвердили правильність розробленої мікропрограми: для режиму множення та для режиму обчислення арифметичної функції отримані значення збігаються з попередньо виконаними ручними обчисленнями. Отже, розроблений мікроалгоритм коректно реалізує задані перетворення даних, а мету лабораторної роботи було досягнуто.

Відповіді на контрольні питання

1. Охарактеризуйте основні способи множення чисел.

Основні способи множення чисел у мікропрограмних пристроях ґрунтуються на послідовному аналізі розрядів множника, формуванні часткових добутків, їх додаванні та зсуві регістрів. Вони відрізняються тим, який операнд зсувається, у якому регістрі накопичується частковий добуток і як організовано регістр результату подвоєної довжини.

- **Перший спосіб** — аналізуються розряди множника, починаючи з молодшого. Якщо поточний розряд дорівнює одиниці, до старшої частини часткового добутку додається множене, після чого пара регістрів часткового добутку зсувається праворуч. Саме цей спосіб використано у цій лабораторній роботі.
- **Другий спосіб** — множене послідовно зсувається ліворуч, а часткові добутки додаються до накопичувача залежно від значення чергового розряду множника.
- **Третій спосіб** — множник і частковий добуток розміщуються у регістрах, які працюють як слово подвоєної довжини; після кожного додавання виконується спільний зсув, що зменшує кількість проміжних пересилань.
- **Четвертий спосіб** — часткові добутки формуються з використанням суматора та керованих зсувів операндів; результат також накопичується поступово, але апаратна організація відрізняється розміщенням регістрів і способом передавання розрядів між ними.

Отже, усі способи реалізують одну математичну операцію, але мають різну структуру операційного пристрою та різну послідовність мікрооперацій.

2. Як забезпечити арифметичний і логічний зсув слів подвоєної довжини?

Слово подвоєної довжини подається парою регістрів: один регістр містить старшу частину числа, а другий — молодшу. Під час зсуву розряд, який виходить з одного регістра, повинен бути переданий до сусіднього регістра. Для цього можна використовувати ознаку переносу `RM_C` або кероване занесення потрібного біта під час мікрооперації зсуву.

При логічному зсуві у звільнений розряд заноситься нуль. При арифметичному зсуві праворуч у старший розряд старшого регістра повторно заноситься знаковий біт, тому знак числа зберігається. У розглянутій системі для цього застосовуються мікрооперації зсуву на кшталт `srl`, `sll`, `sra`, а також перевірка або використання ознаки `RM_C`.

3. Яким чином можна керувати записом інформації в `RM`?

Записом інформації та ознак у `RM` можна керувати спеціальними мікрокомандами завантаження. Основні варіанти:

- `LOAD RM,Z` — встановлення всіх розрядів `RM` у нуль;
- `LOAD RM,NZ` — встановлення всіх розрядів `RM` в одиницю;
- `LOAD RM,FLAGS` — завантаження в `RM` ознак, сформованих після виконання мікрооперації в АЛБ.

Крім того, під час роботи з пам'яттю використовуються керуючі сигнали `R` і `W`, а для формування адреси — `EWn` та `EWL`, які записують відповідно старшу і молодшу частини адреси до регістра адреси.

4. Поясніть призначення директив мікроасемблера.

Директиви мікроасемблера не є мікроопераціями, а задають правила розміщення, початкові значення, конфігурацію зв'язків і допоміжні позначення для трансляції мікропрограми.

Директива	Призначення
ORG	Задає адресу, з якої потрібно розмістити виконавчий код у пам'яті мікрокоманд. Наприклад, <code>ORG 010h</code> .
EQU	Встановлює відповідність між іменем та числовим або мнемонічним значенням. Використовується для констант і псевдонімів.
INCLUDE	Вставляє до мікропрограми текст з іншого файлу, наприклад файл із макросами або спільними підпрограмами.
MACRO	Дозволяє описати власну складену мікрокоманду та надалі використовувати її як звичайну мнемоніку.
DW	Задає початкове значення комірки пам'яті у формі <code>DW <адреса> : <значення></code> .
ACCEPT	Встановлює початковий стан регістрів, зовнішніх пристроїв або параметрів пам'яті, наприклад <code>ACCEPT R4:0FFFh</code> чи <code>ACCEPT RDM_DELAY:3</code> .
LINK	Описує конфігурацію зв'язків між компонентами системи, наприклад підключення умов до входів БМК або поділ регістра адреси сигналами <code>EWH</code> і <code>EWL</code> .

5. Що таке мікроалгоритм, мікропрограма, мікрооперація і мікрокоманда?

Мікрооперація — це елементарна дія, яка виконується апаратними вузлами ЕОМ за один такт або один крок керування: пересилання даних, додавання, логічна операція, зсув, запис до регістра тощо.

Мікрокоманда — це інформаційне слово, яке задає набір керуючих сигналів для виконання однієї або кількох сумісних мікрооперацій, а також може містити інформацію про умову та адресу переходу до наступної мікрокоманди.

Мікропрограма — це послідовність мікрокоманд, виконання яких реалізує певну машинну команду або заданий алгоритм обробки даних.

Мікроалгоритм — це алгоритмічний опис послідовності мікрооперацій, перевірок умов і переходів. На його основі складається мікропрограма.

6. Які мікрооперації в розглянутій системі можна сполучати, а які не можна?

Сполучати можна ті мікрооперації, які виконуються різними вузлами системи та не конфліктують за спільні ресурси. Наприклад, в одному такті можна поєднувати виконання операції в АЛБ із завантаженням сформованих ознак у `RM`, декремент лічильника з перевіркою його стану або підготовку адреси пам'яті з відповідними керуючими сигналами, якщо це не створює конфлікту на шинах.

Не можна сполучати мікрооперації, які одночасно:

- записують різні значення в один і той самий регістр;
- використовують кілька джерел для передавання даних на одну шину;
- вимагають різних несумісних режимів роботи одного пристрою, наприклад одночасного читання і запису однієї комірки пам'яті;

- потребують результату іншої мікрооперації в тому самому такті, якщо для цього не передбачено прямого апаратного шляху;
- задають різні операції АЛБ або різні зсуви для одного й того самого регістра одночасно.

У мікропрограмі сумісні мікрооперації записуються в одних операторних дужках {}, а окремі мікрокоманди всередині них розділяються крапкою з комою.